Weather Stats Analyzer

1. Project Overview

The **Weather Stats Analyzer** is a Python console-based application developed as part of **Phase 1 – Module 2**. The application demonstrates the use of Python collections, user-defined functions, loops, conditional statements, and exception handling to record daily temperatures, perform statistical analysis, convert temperature units, and search recorded data efficiently.

2. Problem Statement

Managing daily temperature records manually can be difficult and may lead to inaccurate calculations. This project automates the process of recording daily temperatures, analyzing weather statistics, converting temperature units, and searching recorded temperatures using Python collections and user-defined functions.

3. Features

Data Collection

- Record daily temperatures
- Validate user input
- Store temperatures in a list

Temperature Analysis

- Find the highest temperature
- Find the lowest temperature
- Calculate average temperature
- Count the number of days above average

Temperature Conversion

- Convert Celsius to Fahrenheit
- Convert Fahrenheit to Celsius

Data Storage & Search

- Store day-wise temperatures using tuples
- Create a set of unique temperatures
- Search whether a specific temperature was recorded

4. Technologies Used

- Python 3
- Visual Studio Code

5. Python Concepts Used

- Functions
- Lists
- Tuples
- Sets
- For Loops
- While Loops
- Conditional Statements
- Exception Handling
- User-defined Functions
- Formatted Output

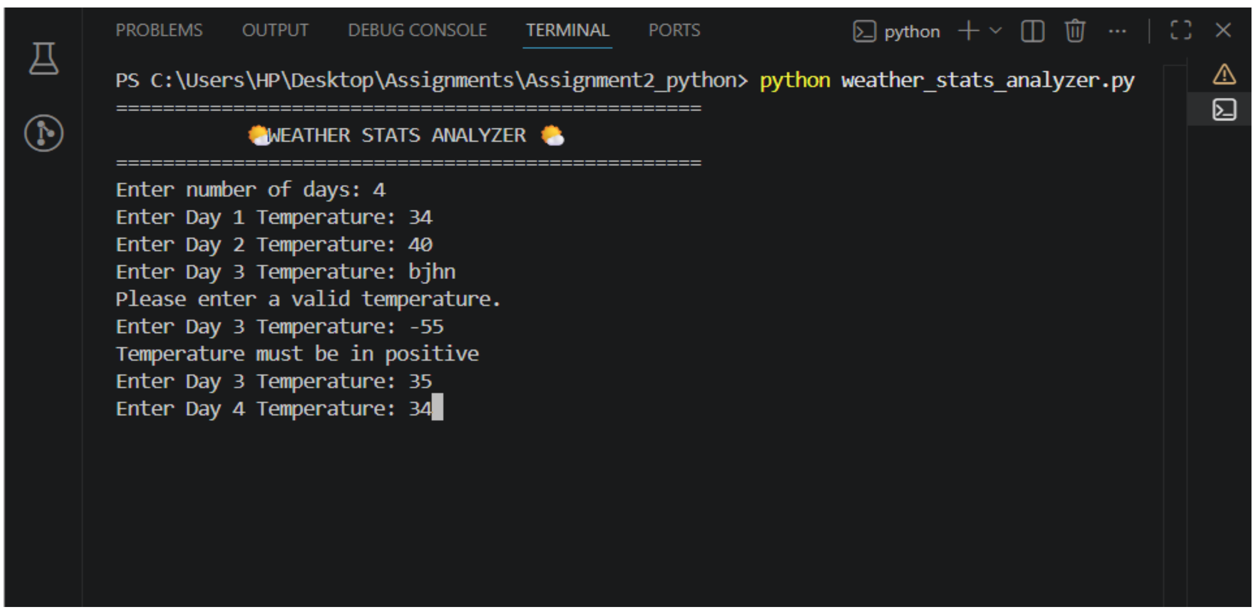
6. Program Workflow

- Start
- Display Welcome Message
- Enter Number of Days
- Record Daily Temperatures
- Display Recorded Temperatures
- Create Unique Temperature Set

- Search for a Temperature
- Perform Temperature Analysis
- Convert Temperature Units
- Display Thank You Message
- End

7. Output Screenshots and Explanation

Welcome Screen



Explanation:

The application starts by displaying a welcome message and prompts the user to enter the number of days for which temperature data will be recorded. It then collects the daily temperatures while validating the user input to ensure only valid numeric values are accepted.

Temperature Data Input

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\HP\Desktop\Assignments\Assignment2_python> python weather_stats_analyzer.py
==================================================
           WEATHER STATS ANALYZER
==================================================
Enter number of days: 4
Enter Day 1 Temperature: 34
Enter Day 2 Temperature: 40
Enter Day 3 Temperature: bjhn
Please enter a valid temperature.
Enter Day 3 Temperature: -55
Temperature must be in positive
Enter Day 3 Temperature: 35
Enter Day 4 Temperature: 34

Daily Temperatures:
Day 1                : 34.00°C
Day 2                : 40.00°C
Day 3                : 35.00°C
Day 4                : 34.00°C
Unique Temperatures: [34.0, 35.0, 40.0]
Enter temperature to search: 36
36.0°C was not recorded.
```

Explanation:

The user enters the daily temperatures, which are stored in a list. The application validates each input before recording it, ensuring that only valid temperature values are accepted.

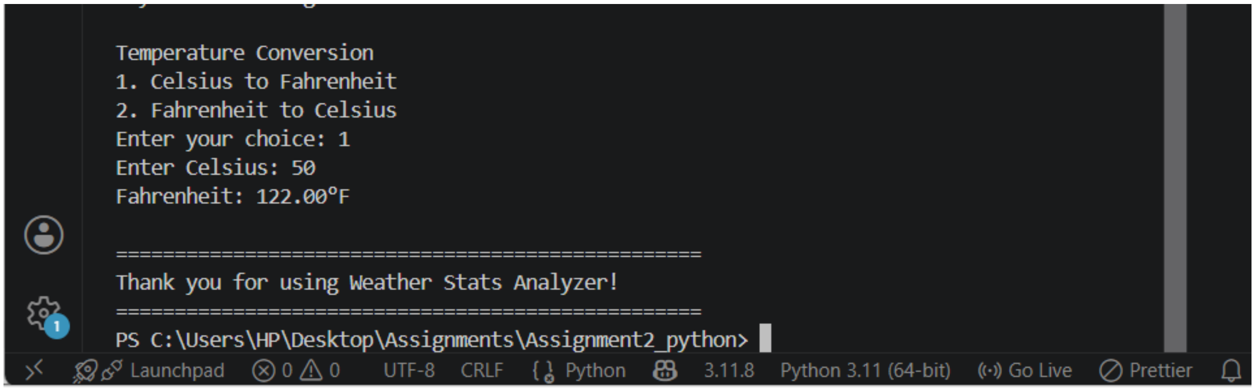
Recorded Temperatures and Search

```
Temperature Analysis
-----------------------------------
Highest Temperature      : 40.00°C
Lowest Temperature       : 34.00°C
Average Temperature      : 35.75°C
Days Above Average       : 1
```

Explanation:

The application displays the recorded temperatures with their corresponding day labels using tuples. It also creates a set of unique temperatures and allows the user to search whether a specific temperature was previously recorded.

Temperature Analysis and Conversions



Explanation:

The application performs temperature analysis by calculating the highest temperature, lowest temperature, average temperature, and the number of days above average. It also converts temperatures between Celsius and Fahrenheit based on the user's choice and displays the converted result before terminating successfully with a thank-you message.

8. Learning Outcomes

- Practiced using Python collections (Lists, Tuples, and Sets)
- Improved modular programming using user-defined functions
- Applied exception handling for input validation
- Performed statistical analysis on real-world data
- Strengthened problem-solving and logical thinking skills
- Learned to organize and present data using formatted output

9. Conclusion

The **Weather Stats Analyzer** was successfully developed using Python fundamentals such as functions, lists, tuples, sets, loops, conditional statements, and exception handling. The application efficiently records daily temperature data, performs statistical analysis, supports temperature conversion, and enables quick temperature lookup using Python collections. This project strengthened my understanding of applying core Python concepts to build a practical, user-friendly, and real-world console application.